Project Report: NERC

Group 10: Thorsten Trinkaus, Laura Komorek, Oliver Jung

April 7, 2026

Contents

1 Introduction **3**

2 Methods **4**
2.1 NLI . . . 4
2.1.1 NLI Approach . . . 4
2.1.2 NLI Formulation . . . 4
2.1.3 Multi-Label Prediction . . . 4
2.1.4 Probing and Fine-Tuning . . . 4
2.1.5 Performance Assessment . . . 4
2.2 Text Generation . . . 5
2.2.1 Data Pre-Processing . . . 5
2.2.2 Model Inference . . . 6
2.2.3 Training Setup . . . 9

3 Datasets and Models **10**
3.1 Datasets . . . 10
3.2 Dataset Analysis . . . 11
3.2.1 Dataset Structure . . . 11
3.2.2 General Overview . . . 11
3.2.3 Comparative Statistics . . . 11
3.2.4 Dataset-Specific Notes . . . 12
3.2.5 Implications for Modeling . . . 12
3.3 Model Choices . . . 12
3.3.1 NLI . . . 12
3.3.2 Text Generation . . . 13

4 Experimental Setup **14**
4.1 NLI . . . 14
4.1.1 Datasets and Data Splits . . . 14
4.1.2 Experimental Pipeline . . . 14
4.1.3 General Setup for Probing and Fine-Tuning . . . 14
4.1.4 Training Configuration . . . 15
4.1.5 Evaluation Settings . . . 15
4.2 Text Generation . . . 15
4.2.1 Dataset Augmentation . . . 15
4.2.2 Experimental Pipeline . . . 16

5 Results and Analysis **18**
5.1 NLI . . . 18
5.1.1 Results Overview . . . 18
5.1.2 Confusion Analysis . . . 18
5.1.3 Error Analysis . . . 18
5.1.4 Robustness Analysis via Masking . . . 19
5.1.5 Type-Based Analysis . . . 19
5.1.6 Discussion of Results . . . 20
5.2 Text Generation . . . 20
5.2.1 CoNLL-2003 . . . 21
5.2.2 FIGER . . . 21
5.3 Ultra-Fine Entity Typing . . . 23
5.4 SpanBERT . . . 24
5.4.1 Pipeline and Training Variants . . . 24
5.4.2 Validation-Based Threshold Selection . . . 24
5.4.3 Final Test Results . . . 24
5.4.4 Dataset-Wise Interpretation . . . 24
5.4.5 Additional Evidence from Logits and Search . . . 25
5.4.6 Summary . . . 25

6 Conclusion **26**

7 Contributions **27**

1 Introduction

Entity typing is the task of assigning semantic types to entity mentions in context. In contrast to classical named entity recognition with only a few coarse labels, modern benchmarks require models to predict fine-grained and often multi-label type sets. This makes the task substantially harder, because systems must combine lexical cues, contextual information, and type dependencies under strong class imbalance.

In this project, we study how model behavior changes with increasing label granularity. We use three benchmark datasets that represent different levels of difficulty: OntoNotes as a coarse-grained single-label setting, FIGER as a fine-grained hierarchical multi-label setting, and Ultra-Fine as a large open-vocabulary setting with thousands of labels. This setup allows us to compare methods under controlled but progressively more challenging conditions.

Methodologically, we investigate three complementary approaches. First, we reformulate entity typing as a Natural Language Inference (NLI) task and score entailment between context and type hypotheses. Second, we model entity typing as text generation with a T5-based sequence-to-sequence setup that predicts label strings directly. Third, we evaluate a SpanBERT-based multi-label classifier with threshold-based prediction and calibration analysis. Together, these approaches cover inference-style reasoning, generative decoding, and direct discriminative classification.

The report is structured as follows. We first describe methods and model choices, then analyze the datasets and their implications for training, and finally present results and error analyses across NLI, T5, and SpanBERT, including cross-dataset comparisons from coarse to ultra-fine granularity.

2 Methods

There are various approaches that can be applied when it comes to entity typing. This chapter explains the approaches that were chosen for this project. Section 2.1 covers a NLI-based approach, while Section 2.2 explains how the task can be framed as a sequence-to-sequence text generation problem.

2.1 NLI

2.1.1 NLI Approach

We reformulate entity typing as a Natural Language Inference (NLI) problem. Instead of directly predicting entity types, the model evaluates whether a hypothesis about an entity is entailed by the input sentence.

2.1.2 NLI Formulation

Given a sentence x and an entity mention e, we construct hypotheses of the form:
"Entity is a [TYPE]."
The model predicts the following:

$$P(y = \text{entailment} \mid x, h_t)$$

2.1.3 Multi-Label Prediction

Since entity typing is a multi-label task, all candidate types are evaluated independently. Labels exceeding a threshold τ are selected.

2.1.4 Probing and Fine-Tuning

We fine-tune pretrained transformer models on the constructed NLI dataset. Each input consists of a premise-hypothesis pair, and the model learns to distinguish between entailment and non-entailment cases.

2.1.5 Performance Assessment

Model performance is assessed using standard multi-label evaluation metrics, including precision, recall, and F1-score.

2.2 Text Generation

Another approach to entity typing is to frame the task as a text generation problem. Here, the goal is to provide the model with the context and the entity mention in a natural language format and have it generate the correct entity types from a predefined set. For this project, we decided on a sequence-to-sequence model that was pre-trained for span reconstruction, which imposes further restrictions on the methods. This section provides an overview of these restrictions and the methods that were used to work around them. This overview includes the data pre-processing, the inference methods, and the training setup.

2.2.1 Data Pre-Processing

Since multiple datasets were utilized for this project, the first step is to convert all datasets to a common format. This format consists of the following:

- **Text**: The input text containing both the context and the entity mention.
- **Mention**: The entity mention that is to be typed. Note that only one mention is included per sample, so dataset samples with multiple mentions per context were converted to multiple samples.
- **Labels**: The gold entity types for the mention. Depending on the dataset, this list can either consist of a single label or multiple labels that are correct for the specific mention.

For inference and fine-tuning, these samples are further converted to prompts that could be processed by the model after tokenization. Since the only specifications imposed on the model so far are that it should be able to generate sequences of natural language text and that it should have been pre-trained for sequence reconstruction, the prompt format has to be as close as possible to the pre-training task. This restriction limits the design of the prompt, as the model is not pre-trained for classification, question answering, or instruction following. Therefore, the context and the list of entity types for the mention have to be converted into a single text sequence, where the entity types should be reconstructed by the model. While early testing of the inference methods, we tested two different prompts:

- **Prompt 1**: *"[TEXT]. '[MENTION]' is a [MASK]."*
- **Prompt 2**: *"[TEXT]. The entity type/s of '[MENTION]' is/are [MASK]."*

where [MASK] is the span that the model should reconstruct. The format of the second prompt depends on the number of types that should be generated for the specific dataset or sample. While the first prompt might be more natural and what one would expect to find in a training dataset for such a model, it does not work for all possible labels (e.g., the type "music" does not fit into the first prompt) and we found that the second

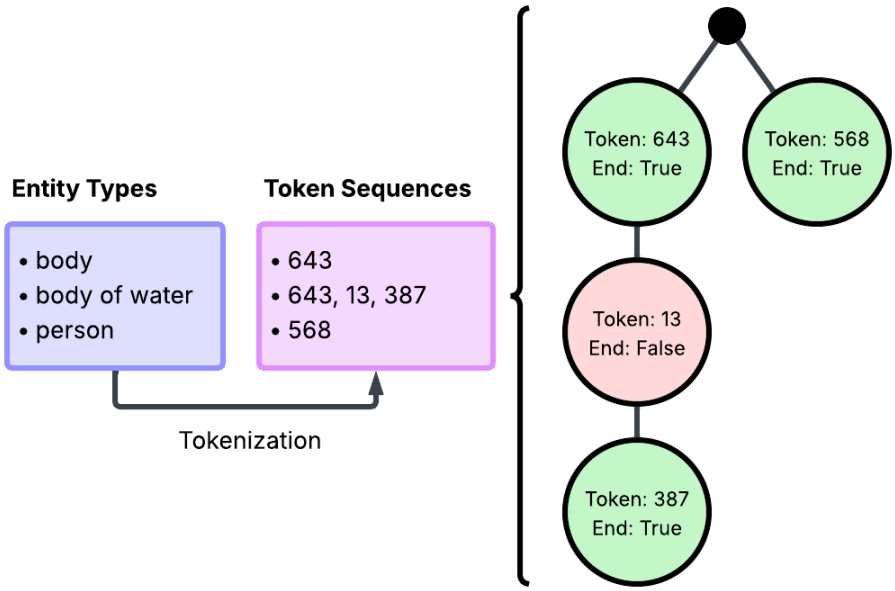

Figure 2.1: Example trie structure for generation-based inference. The trie contains the tokenized entity types "body", "body of water", and "person". Each node contains the token and a boolean indicator if the token is the end of a valid type.

prompt leads to better results, especially for datasets with multiple types per sample. Therefore, this prompt is used for all inference and fine-tuning experiments. An example of such a prompt would be *As the CEO of Apple, Tim Cook unveiled the new iPhone. The entity types of 'Apple' are [MASK].* with the correct entity types being "company" and "technology".

2.2.2 Model Inference

For model inference, two different methods are utilized. First, text generation can be computationally expensive, especially for large datasets. An alternative to generating the entity types for a limited set of candidate types is to compute the likelihood of each candidate given the input prompt. While this method is simple and faster for small candidate sets, as all combinations of entity types and samples can be evaluated with a single forward pass, it has clear drawbacks. For one, the computational cost scales linearly with the number of candidates, since there are NxM combinations of N samples and M candidate types. Additionally, datasets with multiple correct types are hard to evaluate this way, as testing all possible combinations of variable lengths is not feasible. Applying a margin to the most likely candidate can be a workaround, but for better results, inference by text generation is the better choice. This section explains both methods in more detail.

Score-Based Inference

Let $X = \{x_1, x_2, \ldots, x_N\}$ be the set of input prompts and $Y = \{y_1, y_2, \ldots, y_M\}$ be the set of tokenized candidate entity types. For any input prompt $x \in X$, the model defines a conditional probability distribution over the output tokens:

$$P(y_l \mid x) = \prod_{t=1}^{T_l} P(y_{l,t} \mid y_{l,<t}, x), \tag{2.1}$$

where T_l is the number of tokens for the tokenized candidate y_l and $y_{l,<t}$ denotes the sequence of tokens before position t. Taking the log-likelihood of the candidate given the input prompt, a score can be computed and compared across candidates:

$$\phi(x, y_l) = \frac{1}{T_l} \log P(y_l \mid x) = \frac{1}{T_l} \sum_{t=1}^{T_l} \log P(y_{l,t} \mid y_{l,<t}, x). \tag{2.2}$$

Since not all entity types necessarily consist of the same number of tokens, the score is normalized by the number of tokens. This normalization is important to prevent the model from getting biased towards shorter candidates (in terms of number of tokens), as the product only gets smaller with more tokens:

$$\prod_{t=1}^{T_l} P(y_{l,t} \mid y_{l,<t}, x) \leq \prod_{t=1}^{T_k} P(y_{k,t} \mid y_{k,<t}, x) \quad \text{for } T_l > T_k. \tag{2.3}$$

Given prompt $x \in X$ and candidate set Y, the score function ϕ results in the predicted set of entity types

$$\begin{aligned} \hat{y} &= \arg\max_{y_l \in Y} \phi(x, y_l), \\ \hat{Y} &= \{y \in Y \mid \phi(x, y) > \phi(x, \hat{y}) - \epsilon\} \end{aligned} \tag{2.4}$$

where ϵ is a margin to include more than one candidate in the predicted set. If $\epsilon = 0$, the set only contains the most likely entity type.

Generation-Based Inference

Here, the conditional distribution stays the same as in the score-based inference, but instead of computing the likelihood of a small set of candidates, the model freely generates the missing sequence. This makes $\hat{y}$ a sequence of tokens from the model's vocabulary. This method imposes challenges for evaluating the model's ability for entity typing. Allowing generation over the entire vocabulary might lead to reconstructed text that does

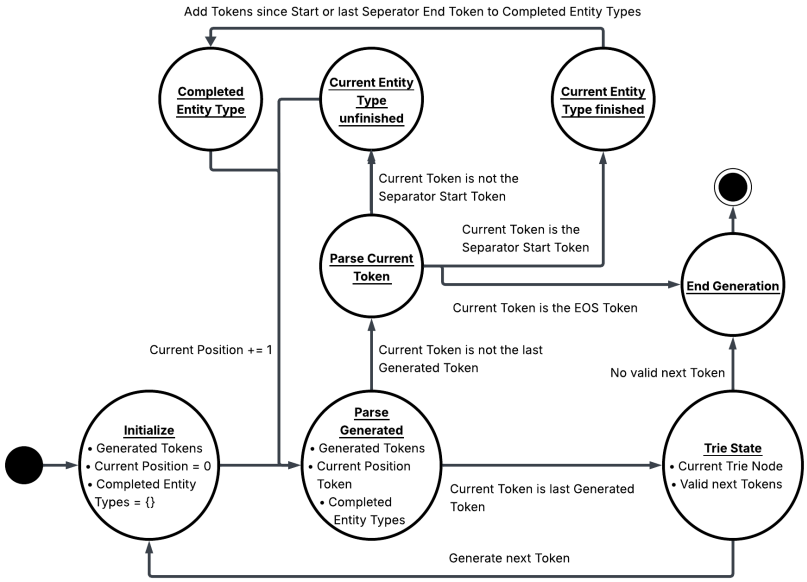

Figure 2.2: Simplified state machine for generation-based inference. The machine tracks the current state of generation and limits the vocabulary of the model after each generation step.

not contain any valid entity types or repeating entity types. Therefore, the model needs to be restricted in the following ways:

- First, the vocabulary of the model needs to be limited to the candidate entity types. With this restriction, it is important that the possible tokens are updated after each generated token, such that the model does not simply rearrange tokens from different candidates to generate invalid text.
- Second, since the model should generate natural text, the generation process requires some sort of separator between the generated entity types. This can be a comma or a word, such as "and", but the model should not be able to generate this separator inside the entity type tokens. This addition makes it necessary to track the generated text and switch between entity types and separators depending on the current state.
- Last, entity types should not be repeated and the model should not generate an unlimited number of entity types, which is why already generated entity types should be identified from the generated tokens.

To realize these restrictions, the vocabulary of possible next tokens is updated after each generated token. For this purpose, all entity types are tokenized and then inserted into a trie data structure. Since entity types can be substrings of other entity types (e.g., "body" and "body of water"), at each node of the trie, not only the token is stored, but

also if this token is the end of a valid entity type. Figure 2.1 shows an example of such a trie for the entity types "body", "body of water", and "person".

The remaining restrictions are implemented by a state machine that parses the previously generated tokens, tracks the current state, and generates a set of valid next tokens. Figure 2.2 visualizes a simplified version of this machine. At each generation step, the machine parses the generated tokens so far one by one and identifies any completed entity types to prevent duplication. Here, a type is marked as completed, if a separator has been generated after the type. If the machine ends up at the last generated token, it generates the next valid tokens based on the current state (i.e., the tokens since the last separator) and the completed entity types by traversing the previously mentioned trie. Valid next tokens are those that continue the current trie node into a direction that has not been completed yet. If the current node ends a valid entity type and this type has not been completed, the start token of the separator is also a valid choice. In the case that the machine does not find any valid tokens (this can also happen because the maximum number of entity types has been reached), the generation is terminated by generating a special end of sentence token. If at least one type has been completed or the current node would end a valid type, this end of sentence token can also be generated by the model as the next token. Should the last generated token be a token from the separator, valid next tokens are restricted to the token that continues said separator.

2.2.3 Training Setup

Fine-tuning the model is pretty straightforward, since the model is already pre-trained for sequence reconstruction. First, to train the model, input samples and the corresponding gold entity types are converted to fit the sequence reconstruction task. The samples are converted to prompts in the same manner as described for model inference. The gold types are concatenated into a single string with the same separator as before. This conversion results in text-target pairs. One such pair could be

TEXT : *"As the CEO of Apple, Tim Cook unveiled the new iPhone. The entity types of 'Apple' are [MASK]."*

TARGET : *"company, technology"*

After tokenization, these pairs can be used to compute a loss function, such as the cross-entropy on the target sequence:

$$\mathcal{L} = -\sum_{t=1}^{T} \log P(y_t \mid y_{<t}, x). \tag{2.5}$$

This loss can then be optimized using standard optimization techniques.

3 Datasets and Models

While the best model depends on the specific methods applied, the datasets listed in this chapter were used for all our experiments. Section 3.1 outlines the datasets, Section 3.2 provides some statistics for each set, and Section 3.3 specifies the model choices for each entity typing approach.

3.1 Datasets

Since the aim of this project is to analyze the performance of different methods and language models for entity typing at different levels of granularity, we decided to use three different datasets that grow in their granularity. The first and most coarse-grained dataset is the CoNLL-2003 dataset by Erik et al. [4]. It contains only four entity types (PER, LOC, ORG, MISC) and provides a type for each token in the input text. While this format works for Named Entity Recognition (i.e., the intended task for this dataset), it is not ideal for the entity typing task at hand. Therefore, each sample is converted to multiple samples, where new samples only contain one mention. Samples without mentions are discarded. This dataset can be directly accessed via Hugging Face [1] and provides training, validation, and test splits.

The second dataset is the FIGER dataset by Ling and Weld [2]. It was accessed via Google Drive [2]. The raw dataset contains Freebase entity types for each mention. Together with the mapping file from the official FIGER repository [3], the entity types were converted to the 113 FIGER types. This dataset is the only one used that contains hierarchical entity types, following a two-level hierarchy. Because the dataset does not contain predefined splits, the samples were split randomly with a ratio of 80% for training, 10% for validation, and 10% for testing.

The last, most fine-grained dataset, is the Ultra-Fine Entity Typing dataset by Choi et al. [1]. This set was accessed directly [4]. It contains more than 10,000 entity types and provides splits for training, validation, and testing that contain crowd-sourced samples. Hence, the training split is very small in comparison to the number of entity types, which is why the dataset provides additional training and validation samples that were obtained via distant supervision. Although these additional splits were concatenated to the crowd-sourced splits, no additional alternations were made to the dataset.

[1] `https://huggingface.co/datasets/eriktks/conll2003`

[2] `https://drive.google.com/file/d/0B52yRXcdpG6MMnRNV3dTdGdYQ2M/view?pli=1&resourcekey=0-n5JGIKP9bdnJ9nbpBYRbXA`

[3] `https://github.com/xiaoling/figer/blob/master/src/main/resources/edu/washington/cs/figer/analysis/types.map`

[4] `http://nlp.cs.washington.edu/entity_type/data/ultrafine_acl18.tar.gz`

3.2 Dataset Analysis

This section follows the structure of the project README and summarizes the dataset properties that are most relevant for model design and evaluation.

3.2.1 Dataset Structure

Although we refer to three benchmark families (OntoNotes, FIGER, Ultra-Fine), the practical setup contains four data sources because Ultra-Fine is split into two substantially different parts:

- Ultra-Fine distant supervision split (`ds_fine_ds`)
- Ultra-Fine crowd-sourced split (`ds_fine_crowd`)

Hence, the effective comparison is between **OntoNotes**, **FIGER**, **Ultra-Fine ds**, and **Ultra-Fine crowd**.

3.2.2 General Overview

Dataset	Task	Granularity	Multi-Label
OntoNotes	Classical NER	Coarse	No
FIGER	Fine-Grained Entity Typing	Fine	Yes
Ultra-Fine	Ultra-Fine Entity Typing	Very Fine	Yes

Table 3.1: Task-level comparison aligned with the project README.

The datasets differ strongly in both scale and label behavior: OntoNotes is small and single-label, FIGER is very large and fine-grained, and Ultra-Fine combines a large label space with heterogeneous supervision.

3.2.3 Comparative Statistics

Dataset	Entities	Unique Labels	Avg Labels/Entity	Multi-word Ratio
OntoNotes	35,089	4	1.00	36.81%
FIGER	4,047,079	91	4.62	0.00%
Ultra-Fine (ds)	3,152,711	4,261	2.18	55.50%
Ultra-Fine (crowd)	5,994	2,519	5.39	50.05%

Table 3.2: Core quantitative statistics from the dataset analysis in the repository README.

From these values, several patterns from the README are confirmed:

- FIGER and Ultra-Fine are much larger than OntoNotes.
- `ds_fine_crowd` is much smaller than `ds_fine_ds`.
- FIGER has only single-token mentions (0% multi-word entities).
- `ds_fine_crowd` has the highest label density per mention.

3.2.4 Dataset-Specific Notes

OntoNotes. OntoNotes corresponds to the standard coarse NER setup with four labels (PER, LOC, ORG, MISC). Since each mention has exactly one label, this dataset acts as the easiest and most stable baseline in our experiments.

FIGER. FIGER extends NER to fine-grained multi-label typing with hierarchical labels (e.g., `/person/actor`, `/organization/company`). Most entities have between one and five labels, while rare cases can reach much higher label counts.

Ultra-Fine. Ultra-Fine uses open natural-language labels and has a long-tail label distribution. The ds split offers scale, while the crowd split provides denser and often more informative labels per mention. This split mismatch is one reason why calibration and thresholding become difficult.

3.2.5 Implications for Modeling

The README-driven analysis motivates dataset-specific strategy choices:

- OntoNotes: focus on robust span-context encoding; calibration is straightforward.
- FIGER: prioritize multi-label consistency and robustness to hierarchical type structure.
- Ultra-Fine: explicit class-imbalance handling and careful threshold calibration are essential.

Overall, the progression from OntoNotes to FIGER to Ultra-Fine reflects increasing granularity, label ambiguity, and long-tail effects, which explains why methods transfer only partially across datasets.

3.3 Model Choices

3.3.1 NLI

We use transformer-based models:

- RoBERTa-base for CoNLL-2003
- DistilRoBERTa for FIGER and Ultra-Fine

The RoBERTa-base model is used for smaller datasets to maximize performance, while the DistilRoBERTa model is used for larger datasets to reduce computational cost.

3.3.2 Text Generation

For the text generation approach, we utilized the T5-base model by Raffel et al. [3], that was accessed via Hugging Face [5]. This model was pre-trained for sequence reconstruction, and therefore, fits the requirements for the generation-based inference methods. Out of the box, the model provides special tokens in the form of "<extra_id_0>", "<extra_id_1>", etc. that were used during pre-training to mask spans for reconstruction. To mark the reconstructed span, these tokens are also part of the target sequences. For example, the following text-target pair could have been used for training:

TEXT : *"As the CEO of Apple, <extra_id_0> unveiled the new iPhone. The entity types of 'Apple' are <extra_id_1>."*

TARGET : *"<extra_id_0> Tim Cook <extra_id_1> company <extra_id_2>"*

[5] `https://huggingface.co/google-t5/t5-base`

4 Experimental Setup

This chapter details the experimental setup for all entity typing approaches. Section 4.1 describes the setup for the NLI-based method, while Section 4.2 explains the setup for the text generation approach.

4.1 NLI

4.1.1 Datasets and Data Splits

We perform our experiments on the three previously mentioned datasets: CoNLL-2003, FIGER and Ultra-Fine Entity Typing, which differ in their granularity.

For each dataset, we use the standard train, validation, and test splits which were already provided.

This consistent setup ensures comparability across the datasets despite their differing characteristics.

4.1.2 Experimental Pipeline

Our experimental pipeline consists of three main stages:

1. **NLI Data Construction:** Each dataset is converted into a set of premise-hypothesis pairs using a predefined template.

2. **Model Fine-Tuning:** Transformer models are then fine-tuned on the constructed NLI training data.

3. **Inference and Evaluation:** Our fine-tuned models are evaluated on the test set by computing entailment scores for all candidate types.

4.1.3 General Setup for Probing and Fine-Tuning

The Fine-Tuning is performed using labeled premise-hypothesis pairs derived from our training data. The models learn to distinguish between "ENTAILMENT" (correct type) and "NOT ENTAILMENT" (incorrect type).

To analyze model representations, we evaluate the pretrained (non-fine-tuned) models first. This allows us to interpret the extent to which entity type information is already encoded in the pretrained representations versus which are learned during the fine-tuning.

4.1.4 Training Configuration

All models are fine-tuned using a binary cross-entropy objective over entailment labels. The training is performed using mini-batch gradient descent.

However, given the large label spaces in the FIGER and Ultra-Fine datasets, we apply negative sampling. Using all possible negative labels would lead to an extreme class imbalance.

For each positive instance, we randomly sample 5 negative labels. This not only reduces computational cost but also should help maintaining a more balanced ratio between our positive and negative examples.

For the CoNLL-2003 dataset, due to the smaller label space, all negative labels are included without additional sampling.

4.1.5 Evaluation Settings

We evaluate model performance using standard metrics for multi-label classification, including precision, recall and F1-score. Specifically, we report both the micro-averaged and the macro-averaged variants of these metrics.

Micro-average metrics show predictions across all instances and are therefore dominated by the majority class. In contrast, macro-averaged metrics compute scores independently for each class and then average them giving equal weight to all of the classes regardless of their frequency.

This distinction is especially important for our setting, as the NLI formulation still introduces a strong imbalance between "ENTAILMENT" and "NON ENTAILMENT" examples, even after sampling.

All metrics are computed using the implementation proved by the scikit-learn library.

4.2 Text Generation

All experiments for the text generation approach were performed with Python version 3.14.3. This section provides a detailed overview of the dataset augmentations, the experimental pipeline, the inference and fine-tuning configurations, and the metrics used for evaluation.

4.2.1 Dataset Augmentation

To fit the task at hand and the model used, some changes were made to the datasets. First, the entity types from the original datasets had to be converted into a more natural language format. For example, the types “PER” from the CoNLL-2003 dataset and “/person/artist” from the FIGER dataset were converted to “person” and “artist”, respectively. Additionally, underscores were replaced with whitespaces (e.g., “body_of_water” to “body of water”). Specifically in the case of FIGER, the dataset contains the types “livingthing” and “living thing”, which were merged into “living thing”.

For inference and fine-tuning, the sample size was reduced by randomly sampling from the datasets to save time and computational resources. We fixed the sample size to a maximum of 500,000 samples for training, 50,000 for validation, and 25,000 for testing. In the case of Ultra-Fine Entity Typing, the crowd-sourced splits were used to their full extent, while increasing training and validation splits with the distantly supervised samples. Here, the remaining samples for the 500,000 training and 50,000 validation samples were randomly sampled, half from the entity linking and half from the head word supervision splits.

4.2.2 Experimental Pipeline

The experiments were performed in the following steps:

1. The datasets were pre-processed to convert them into the common format and to get the prompt-target pairs for inference and fine-tuning.
2. The next step was to perform zero-shot inference on the test sets to evaluate the performance of the pre-trained model.
3. Third, the model was fine-tuned on the respective training and validation sets for each dataset.
4. The inference step was repeated with the fine-tuned model to evaluate the performance improvements through fine-tuning.
5. Finally, we performed robustness and generalization tests to analyze the model behavior further.

Inference

For CoNLL-2003, both score-based and generation-based (with ',' as the separator) inference were performed and compared. For the other two datasets, only generation-based inference was viable due to the number of entity types. Model generation was limited to 8 entity types and performed via beam search with a beam width of 5 to find the best sequence instead of the best token at each generation step. To test the robustness of the model and to analyze the reliance on specific input features (e.g., the actual entity mention instead of the context), we also ran inference with two masking strategies, one with masked mention and one with masked context. These additional tests produced three types of prompts for each sample:

Original Prompt :
"[TEXT]. The entity type/s of '[MENTION]' is/are <extra_id_0>."
Masked Mention :
"[TEXT BEFORE MENTION] <extra_id_1> [TEXT AFTER MENTION].
The entity type/s of '<extra_id_1>' is/are <extra_id_0>."
Masked Context :
"<extra_id_1> [MENTION] <extra_id_2>.
The entity type/s of '[MENTION]' is/are <extra_id_0>."

We applied <extra_id_1> and <extra_id_2> as masks, since the model is already familiar with these tokens and internally tries to reconstruct the corresponding sequences. For evaluation, only the reconstruction for <extra_id_0> was considered.

As FIGER is hierarchical, with a few coarse-grained type classes, we also evaluated the performance of the model, in the case of already provided coarse-grained types. An example for this test would be *"The entity types of 'Kurt Cobain' are person, <extra_id_0>."* with the correct fine-grained types being "artist" and "musician". This test is interesting, since predicting the coarse-grained types first is comparable to the CoNLL-2003 dataset.

Fine-Tuning

Fine-tuning was performed for 3 epochs in the case of CoNLL-2003 and for 2 epochs for the other two datasets. As specified by the model's documentation, the learning rate was set to a slightly higher value of $3e-4$. As the optimizer, Adafactor was used, which is a memory-efficient variant of Adam. Training itself was performed via Seq2SeqTrainer from the transformers python package [1].

Generalization

Both FIGER and Ultra-Fine Entity Typing contain the coarse-grained types from the CoNLL-2003 dataset (with the exception of "misc"). This overlap allowed to test for the ability to predict coarse-grained types after fine-tuning on the more fine datasets.

Evaluation Metrics

For evaluation, we used the same metrics as for the NLI-based approach, which are precision, recall, and F1-score.

[1] `https://pypi.org/project/transformers/`

5 Results and Analysis

5.1 NLI

5.1.1 Results Overview

Dataset	Micro F1	Macro F1
CoNLL-2003	0.75	0.43
FIGER	0.69	0.41
Ultra-Fine	0.99	0.50

Table 5.1: Overall model performance across datasets.

Across all datasets, we observe a substantial gap between micro and macro F1 scores. While micro F1 suggests strong overall performance, macro F1 reveals significant weaknesses in handling the minority class.

This indicates that the models are strongly biased towards the majority class ("NOT ENTAILMENT"), which dominates the dataset due to the NLI formulation.

5.1.2 Confusion Analysis

To better understand the models behaviors, we analyze confusion matrices for each dataset.

Across all datasets, a consistent pattern emerges:

- The majority of predictions are "NOT ENTAILMENT"
- False negatives dominate the error distribution
- False positives are extremely rare or even non-existent

This confirms that the model systematically fails to detect positive entailment cases and instead is defaulting to the negative class.

5.1.3 Error Analysis

We further analyze erros by separating false positives (FP) and false negatives (FN).

Across all datasets, we observe:

- Almost no false positives

- A large number of false negatives

This imbalance indicates that the model is highly careful in predicting entailment. While this avoids incorrect positive predictions, it also leads to a severe drop in recall for the entailment class.

5.1.4 Robustness Analysis via Masking

To investigate the models reliance on input features, we perform robustness tests using simple masking strategies.

We apply two types of masking:

- **Entity Masking**: Named entities (e.g., PERSON) are replaced with [MASK]
- **Span Masking**: Entire noun phrases are masked

We then measure robustness as the proportion of unchanged predictions after masking.

CoNLL-2003 The model shows 100% robustness for both masking strategies.

However, this result is misleading. The model already fails to predict any positive entailment examples correctly, classifying all instances as "NOT ENTAILMENT". As a result, masking does not change any of the predictions.

Therefore, the observed robustness reflects a degenerate behavior rather than true robustness.

FIGER For FIGER, robustness drops to 0%, meaning that nearly all predictions change after masking.

The error analysis shows that all previously predicted "ENTAILMENT" cases are changed to "NOT ENTAILMENT". This indicates that the model heavily relies on specific lexical cues in the hypothesis.

Once these cues are removed, the model is unable to identify entailment.

Ultra-Fine For Ultra-Fine, the model again shows 100% robustness.

Similar to CoNLL-2003, this is due to the strong bias towards predicting "NOT ENTAILMENT". The model produces almost exclusively negative predictions, resulting in unchanged outputs after the masking.

Therefore, the robustness does not reflect stability, but rather the model's inability to detect positive entailment in the first place.

5.1.5 Type-Based Analysis

We further analyze performance across entity types.

Since each mention is paired with multiple candidate types, type frequencies are approximately uniform. Therefore, performance differences across types cannot be explained by frequency effects.

Instead we observe that:

- Some types achieve consistently higher F1 scores
- Other are significantly harder to predict

This suggests that the model captures certain semantic categories better than others, while fine-grained distinctions remain challenging.

5.1.6 Discussion of Results

Overall, the results reveal several key points:

- The NLI formulation introduces a strong class imbalance
- Models are heavily biased towards predicting "NOT ENTAILMENT"
- Positive entailment cases are rarely detected
- High robustness scores can be misleading and may reflect degenerate behavior
- Performance varies across entity types, highlighting challenges in the fine-grained typing

These findings indicate that while the NLI-based approach is effective for generating training data, it introduces systematic biases that significantly impact the models performances.

5.2 Text Generation

Dataset	Zero-Shot Micro F1	Zero-Shot Macro F1	Micro F1	Macro F1
CoNLL-2003	0.31	0.23	0.95	0.95
FIGER	0.18	0.11	0.91	0.84
Ultra-Fine	0.03	0.00	0.27	0.02

Table 5.2: Overall T5 performance across datasets.

Table 5.2 lists the overall performance of T5 for both zero-shot inference and repeating inference after fine-tuning. The metrics show that the task becomes more difficult with increasing granularity, which is expected, and that micro and macro F1 scores are close together for all datasets and both inference methods. Since micro F1 is always slightly higher, the model seems to perform marginally better on the more frequent types, which is not surprising, as the (especially more fine-grained) datasets show clear type imbalances, with common types, such as “location” being more frequent than rare or hard to predict types. This section provides a more detailed analysis for each individual dataset.

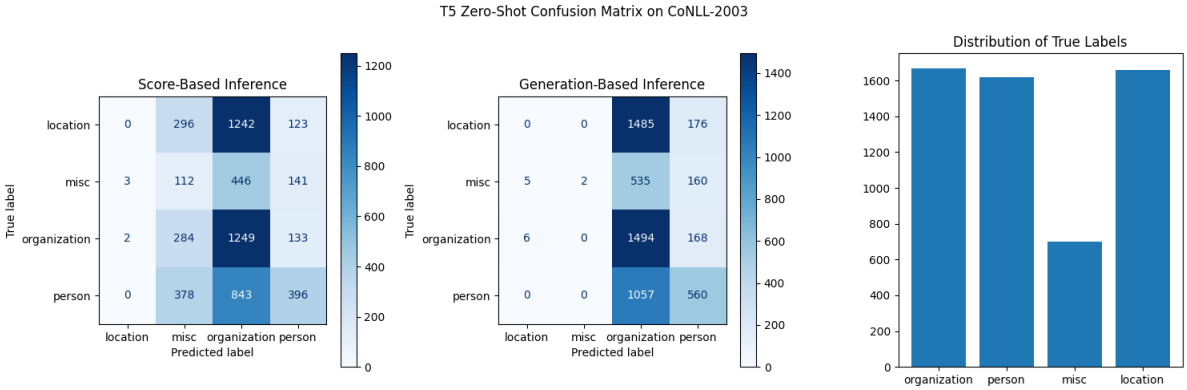

Figure 5.1: Confusion matrix for zero-shot inference on CoNLL-2003.

5.2.1 CoNLL-2003

While the zero-shot performance might seem ok at first, the confusion matrix in Figure 5.1 reveals that the model mostly defaults to predicting a single type, which luckily is the most frequent type in the dataset. Therefore, the micro F1 score is comparable to the baseline of always predicting this type. That the model is not able to predict any relevant type and defaults to one of them is further confirmed by testing masking the mention and the context, which lead to almost the same results (micro F1 of 0.31 and 0.31, and macro F1 of 0.18 and 0.19, respectively). This behavior suggests that either the model is not able to understand the task at all or that the dataset does not provide enough information for the model to make informed predictions. Interestingly, the model has a higher precision for the type "person" when masking the context, which suggests that the context might be misleading or confusing for the model. An example that backs this claim is the text *"C. Spearman c Moin Khan b Wasim 0"* with the mention "C. Spearman" and the correct type "person". From the context, this short text from some sort of list, might be confusing to predict even for a human, but after masking the context, only the relevant name remains.

Fine-tuning the model took about 20 minutes and resulted in a significant performance boost, with most samples being correctly classified. Performing masking after fine-tuning shows a significant drop in performance (both F1 scores around 0.4 for mention and 0.77 for context masking). This drop indicates that the model relies on both the mention and the context to make its predictions, which is a good sign for the model's understanding of the semantics of the text. The difference between the two masking strategies makes sense when looking at the dataset, as some samples just don't contain enough context to make a guess without the mention (e.g., the text *"Nadim Ladki"* with the mention "Nadim Ladki").

5.2.2 FIGER

The distribution of true and predicted types in Figure 5.2 show a clear imbalance in the dataset, making macro F1 the relevant metric here. It also shows that, even if not as

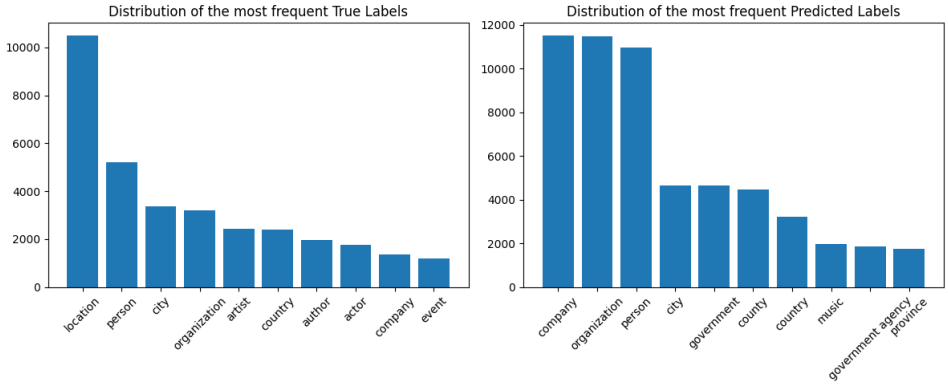

Figure 5.2: Distribution of FIGER true and predicted types.

extreme, the model still default to predicting some types significantly more often than others. One of the most frequent predictions is organization, which is also the most frequent type for CoNLL-2003. This consistency might indicate a bias in the model and that the model's pre-training did not prepare it well for the entity typing task. Comparing the macro F1 score to the baseline of predicting the three most frequent types (0.01), shows that, overall, the model is still able to make some informed predictions. With both masking strategies, the micro F1 score only drops slightly (0.13 for mention and 0.15 for context masking), suggesting that the model is not relying on specific input features for the most frequent types in the dataset. However, masking the mention drops the macro F1 score to half of the original score (0.05), while masking the context does not influence the macro score, which indicates that the model overly relies on the mention for predictions instead of the semantic context.

Since FIGER is hierarchical, we also tested what would happen if the model already had the coarse-grained types within the prompt. This test is interesting, as predicting these types first is comparable to the CoNLL-2003 dataset, which required less training data and resulted in faster training and inference. We observed that the provided types would lead the model towards predicting other coarse-grained types, instead of more specific fine-grained types. For example, if the prompt contains the type "organization", the model would prefer "person" over more fine types, such as "company" or "sports league". This behavior suggests that the model can not utilize the provided types and that, to make use of them, further alternations to the prompts would be necessary.

Fine-tuning the model on the dataset took about 8 hours, with the increased performance being comparable to the performance increase of CoNLL-2003. Masking the mention leads to significant drops in both F1 scores (0.49 for micro and 0.32 for macro). Since the macro F1 score drops to less than half of the non masking score, the model still relies heavily on the mention itself. Masking the context also leads to drops (0.76 for micro and 0.66 for macro), but they are not as extreme. This difference suggests that the model, while relying on the mention, also utilizes the context to some extent. Some of this drop might also be explained by the dataset, as for some samples, the context adds more information than would be necessary for the golden types. For ex-

ample, the text *"He was the ninth child and sixth son of George III and Charlotte of Mecklenburg-Strelitz ."* with the mention "Charlotte of Mecklenburg-Strelitz", only has the type "person". However, when relying on the context, the model includes all types that apply to the context, which are "person", "monarch", and "politician", which are not that bad, but still incorrect.

Lastly, we tested if a model fine-tuned on the fine-grained FIGER dataset would still be able to predict the coarse-grained types from CoNLL-2003, as all types (with the exception of "misc") are included in FIGER. While precision and recall for the type "person" are both around 0.85-0.9, the types "location" and "organization" underperform compared to the model fine-tuned on CoNLL-2003. We looked into the results, and found a possible explanation for this behavior. CoNLL-2003 contains a lot of samples, where countries perform the role of an organization (e.g., "Japan" as in the japanese football team). We have already seen that the model predictions are influenced the most by the mention, not the context. Therefore, the model fine-tuned on FIGER recognizes these mentions as locations. This behavior, together with the observation that CoNLL-2003 often lacks context, might explain the mixed performance for the two types.

5.3 Ultra-Fine Entity Typing

Since the test set is very small, it is not possible to test all of the roughly 10,000 types. This mismatch directly leads to a very large type imbalance, which has to be considered when interpreting the results. When masking the mention, the very bad performance of the model drops even further (0.01 for micro F1). However, masking the context does not lead to a significant drop in macro F1 (0.004 for both with and without masking). This result is consistent with the previous datasets, as the model seems to not fully understand the task, depends on the mention, and introduces a strong bias towards some label types. The problem is made even worse by the fact that the dataset contains a lot of indirect mentions, such as "he" or "it", which, by themselves, can not be linked to any type accurately.

Fine-tuning, while still being lackluster, shows a significant improvement. Macro F1 stays very low, but that is to be expected, as macro F1 is computed over the F1 scores for each type, and most types are not present in the test set, which leads to a lot of zero F1 scores. The micro F1 score, performs much better, but the sheer number of types would possibly require larger test sets and more training, to better understand the model's performance on this dataset. Masking showed no new insights.

Generalization tests showed that the fine-tuned model performed similar to the FIGER case. Precision and recall stay in the same range of around 0.88 for the type "person", while the other two types underperform. This finding further suggests that, while the performed training was not enough to get good results on the Ultra-Fine Entity Typing dataset, the model still learned some coarse-grained type information within the 10,000 dataset types.

5.4 SpanBERT

This section summarizes the SpanBERT-based multi-label classification pipeline and integrates the final results stored in `results/spanbert_probing/*`.

We use `SpanBERT/spanbert-base-cased` and cast entity typing as multi-label sentence-pair classification between the full sentence and the mention span. The model outputs one logit per label, optimized with `BCEWithLogitsLoss`; final predictions are obtained by applying a global threshold tuned on the validation split.

5.4.1 Pipeline and Training Variants

The preprocessing script converts OntoNotes, FIGER, and Ultra-Fine into a unified JSONL format with the fields `sentence`, `start`, `end`, and `labels`. Fine-tuning is performed separately per dataset.

For OntoNotes and FIGER, standard multi-label BCE training was stable and produced strong results. Ultra-Fine required additional imbalance handling: an unweighted run collapsed to trivial predictions, so we retrained with weighted BCE (`-use-pos-weight -pos-weight-max 50`) and then performed a finer threshold search in the negative range.

5.4.2 Validation-Based Threshold Selection

Thresholds were selected on validation using `f1_micro` as selection metric:

- OntoNotes: best threshold 0.20, with nearly identical alternatives at 0.25–0.40.
- FIGER: best threshold 0.30, while nearby values (0.20, 0.25, 0.40) were almost tied.
- Ultra-Fine: best threshold -0.045; even small shifts changed calibration noticeably.

This confirms that thresholding is mostly a minor calibration step on OntoNotes/FIGER, but a critical component on Ultra-Fine.

5.4.3 Final Test Results

5.4.4 Dataset-Wise Interpretation

OntoNotes. SpanBERT achieves very high and balanced performance (micro and macro F1 are almost identical). The predicted label density is almost exactly one label per sample, matching the single-label nature of OntoNotes. This indicates both strong discrimination and good calibration.

Metric	OntoNotes	FIGER	Ultra-Fine
Threshold	0.2000	0.3000	-0.0450
Micro Precision	0.9732	0.9544	0.0574
Micro Recall	0.9726	0.9380	0.3847
Micro F1	0.9729	0.9462	0.0999
Macro Precision	0.9721	0.9042	0.0008
Macro Recall	0.9734	0.8640	0.0143
Macro F1	0.9727	0.8816	0.0015
Avg. Gold Labels / Example	1.0000	2.3877	5.3719
Avg. Predicted Labels / Example	0.9994	2.3466	35.9965

Table 5.3: comparison of final SpanBERT test metrics across datasets.

FIGER. Results remain strong under a fine-grained multi-label setup. Precision and recall are both high, and predicted label density closely matches the gold density. The drop from micro F1 (0.9462) to macro F1 (0.8816) shows that rare FIGER types are still harder, but the overall model behavior is robust.

Ultra-Fine. Ultra-Fine remains the hardest setting by a large margin. Weighted BCE prevented full collapse and enabled non-trivial recall, but calibration is still poor: the model predicts about 36 labels on average for samples with about 5.37 gold labels. This overprediction yields recall-heavy but low-precision behavior and very weak macro F1.

5.4.5 Additional Evidence from Logits and Search

The stored probing metrics support this interpretation. OntoNotes and FIGER show large positive max-logits on average (about 5.77 and 6.99), while Ultra-Fine has a very narrow top-logit range (average max logit about 0.094). This explains why Ultra-Fine decisions are highly threshold-sensitive and why a slightly negative threshold was required for best micro-F1.

5.4.6 Summary

Across datasets, SpanBERT provides a strong baseline for coarse and medium-granularity entity typing, with excellent results on OntoNotes and very competitive results on FIGER. For Ultra-Fine, the current setup demonstrates that class-imbalance reweighting is necessary but not sufficient. The next improvements should target calibration directly, e.g., label-wise thresholds, frequency-aware decision rules, or post-hoc calibration methods.

6 Conclusion

This report presented our methods, experiments, and results for the entity typing task. We analyzed three different models (with their own approaches to entity typing) on three different datasets to explore the capabilities of pre-trained language model for the said task. Across all experiments, we observed that the performance heavily depends the approach and the dataset granularity.

NLI-based methods treated entity typing as a binary classification problem, which is simple, but introduced a strong class imbalance between entailment and non-entailment. This imbalance caused biases in the predictions towards the majority class (non-entailment). Therefore, this method has proven to be unreliable for entity typing fine-grained datasets.

Text generation with T5 performed bad on zero-shot inference, indicating no real capabilities for entity typing without fine-tuning. However, after fine-tuning, the model showed great performance and even some generalization capabilities to coarse-grained types after training on fine-grained types. Nevertheless, this method struggled with dataset imbalances and relied more on the mention itself than on the context, which indicates that the model does not predict right for the right reasons.

SpanBERT performed the best out of the three models and showed good performance across CoNLL-2003 and FIGER. As with T5, performance on the Ultra-Fine Entity Typing dataset was very bad, which might be explained to some extent by the dataset itself. Since FIGER performs slightly worse than CoNLL-2003, it still has to be assumed that the performance decreases with higher granularity, but further tests would be necessary to confirm this hypothesis.

The hypothesis from our project proposal were almost all confirmed by the results. First, model performance decreases with increasing granularity. Second, T5 performed well on multi-word entities, as we could not find any significant decrease in performance. Further tests should be performed to confirm this finding, especially in comparison to the other models. Third, the NLI approach seems to be robust towards granularity, even if the overall performance is biased towards the majority class. Lastly, T5 experiments have shown that the model relies more on the mention itself, rather than any semantic understanding of the context, but they have also shown that the model is able to incorporate the context into the predictions.

The only hypothesis that could not be extensively tested was the fine-tuned models showing confusion among semantically similar types. For this hypothesis, further tests would be necessary and a reasonable next step. Additionally, the performance of our methods should be tested against more fine-grained datasets that do not suffer from the same type imbalance, especially in the granularity range of Ultra-Fine Entity Typing, but such datasets could potentially be hard to find. Hence, focus should be put on SpanBERT, as the best performing model, in search for further improvements.

7 Contributions

For this report, the bottom of each page lists the contributors that worked on the respective page. This last chapter provides a summary of the further contributions made by each group member:

- **Laura Komorek**: Creating pre-processing scripts for the NLI approach. Preparing the NLI evaluation setup, such as creating the inference pipeline. Implementing the NLI-based probing and fine-tuning approaches as well as investigating the effects of masking strategies. Creating the evaluation scripts specifically for NLI. Those contain scripts for Precision, Recall and both micro and macro F1-scores. Analyzing results from the NLI-based approaches, such as error and confusion analysis including bias and pattern predictions, as well as the probing and robustness analysis. The results and findings of these steps can be found in the report sections written by Komorek.

- **Thorsten Trinkaus**: Gathering and pre-processing datasets. This pre-processing includes mapping entity types and splitting the datasets into training, validation, and test sets. Additionally, the complete text generation approach, from planning, task specific data pre-processing, implementation of inference and fine-tuning, to the analysis of the results. Hence, all report sections related to these steps were written by Trinkaus, as well as chapter summaries and the final conclusion. Another responsibility was to setup the GitHub repository.

- **Oliver Jung**: My main contribution to the project focused on two key areas: dataset analysis and the implementation of SpanBERT-based experiments.

  First, I conducted a comprehensive analysis of the datasets (OntoNotes, FIGER, and Ultra-Fine). This included examining label distributions, dataset sizes, granularity differences, multi-label characteristics, and the prevalence of multi-word entities. I also created visualizations and comparisons to better understand the structural differences between the datasets.

  Second, I was responsible for adapting the project pipeline to SpanBERT. This involved designing and implementing a preprocessing pipeline tailored to span-based classification, converting the datasets into a suitable format, and setting up a multi-label classification framework. I also implemented and executed fine-tuning experiments with SpanBERT to evaluate its performance across datasets with different levels of granularity.

Bibliography

[1] Eunsol Choi, Omer Levy, Yejin Choi, and Zettlemoyer. Ultra-fine entity typing. In *Proceedings of the ACL*. Association for Computational Linguistics, 2018.

[2] Xiao Ling and Daniel S. Weld. Fine-grained entity recognition. In *Proceedings of the Twenty-Sixth AAAI Conference on Artificial Intelligence*, 2012.

[3] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.

[4] Erik F. Tjong Kim Sang and Fien De Meulder. Introduction to the CoNLL-2003 shared task: Language-independent named entity recognition. In *Proceedings of the Seventh Conference on Natural Language Learning at HLT-NAACL 2003*, pages 142–147, 2003.